

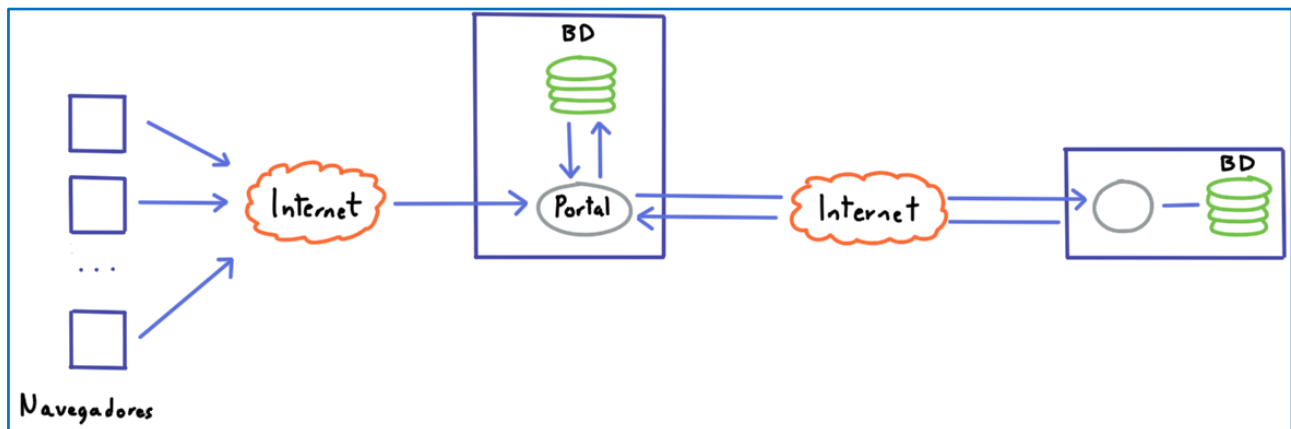
TEMA 5: LENGUAJES DE INTERCAMBIO DE DATOS: XML Y JSON

INTRODUCCIÓN

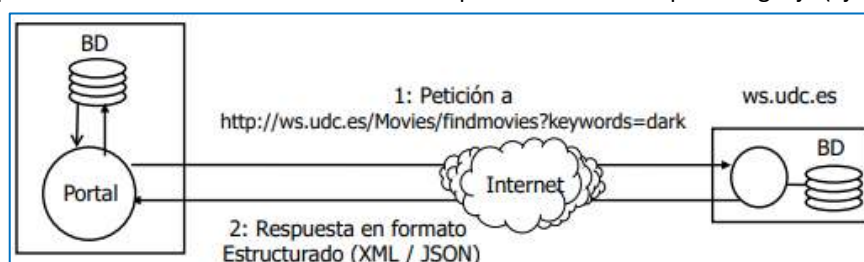
- Se quiere exponer la capa modelo creada (tema 3) como un servicio que pueda ser utilizado por aplicaciones remotas
- Aplicaciones remotas
 - Pueden residir en otras máquinas o acceder a la capa modelo a través de Internet
 - Pueden estar escritas en cualquier lenguaje de programación
 - Proporcionarán su propia interfaz gráfica (si la tienen) y pueden utilizar otras bases de datos o aplicaciones adicionales además de la nuestra
- Necesarios lenguajes o formatos de datos robustos → deben expresar y transmitir información compleja entre sistemas heterogéneos
 - Ej. XML, JSON, YAML

EJEMPLO

- Un portal externo desea ofrecer el servicio de búsqueda y visión de películas
 - Tiene su propia capa de interfaz gráfica
 - Interfaz gráfica integrada con sus propios servicios
 - Añade críticas de las películas escritas por los usuarios
 - Cuando los usuarios del portal buscan películas, el portal invocará findMovies() en nuestro servicio web (que invocará a nuestra capa modelo)
 - Al mostrar los resultados, añadirá las críticas a cada película obtenidas de su BD
 - Ofrece descuentos a algunos usuarios
 - El portal calculará el precio a mostrar a dicho usuario restándole el descuento al precio devuelto
 - Etc.



- Se supone que nuestro modelo da más información sobre cada película
 - Director/es
 - Principales actores
 - Fecha de estreno
 - Género → 1 o varios de entre una lista de valores posibles (comedia, romance, thriller, etc.)
- Opción → nuestro servicio expone una URL a la que es posible pasarle como parámetros (método GET de HTTP) las keywords de búsqueda
- El portal remoto invocará dicha URL, nuestro servicio invocará a la capa modelo para obtener los datos y los devolverá al portal en un formato estructurado fácil de parsear desde cualquier lenguaje (ej. XML o JSON)



- Recordatorio HTTP
 - HTTP es un protocolo de nivel de aplicación, mediante el cual un cliente y un servidor pueden intercambiar fácilmente información
 - HTTP no tiene nada que ver con HTML
 - Desde el punto de vista de HTTP, HTML, XML o JSON son simplemente texto que intercambian cliente y servidor
 - HTTP soporta diversos métodos de invocación de URLs
 - Más popular → GET → utilizado para acceder a los datos del recurso referenciado
 - Las peticiones GET soportan paso de parámetros en la URL con la sintaxis **?param1=valor¶m2=valor2**

CAMPOS DE APLICACIÓN

- Intercambio de datos entre aplicaciones heterogéneas → (la asignatura se centra en esta parte)
- Generación de vistas (HTML, PDF, etc.) a partir de documentos de datos
- Configuración de aplicaciones
- Bases de datos
- etc.

EL LENGUAJE XML

¿QUÉ ES XML?

- XML (eXtensible Markup Language)
 - Lenguaje de tags (similar en sintaxis a HTML)
 - Estandarizado por el W3C
- Es extensible
 - No impone un conjunto de tags, sólo unas pocas normas sobre cómo usarlos
 - Los tags se abren (**<tag>**) y se cierran (**</tag>**) y en medio pueden tener otros tags anidados
 - Todos los documentos tienen un tag raíz
 - Los tags pueden tener atributos
 - etc.
- Permite expresar información estructurada y fácilmente parseable
- Objetivo del tema → **entender los fundamentos básicos de XML necesarios para la asignatura**

INFORMACIÓN EN XML

- Uso de XML para expresar información de películas
 - El texto XML es el que devolvería la URL de nuestro servicio y que parsearía el portal del ejemplo de la introducción del tema

```

<?xml version="1.0" encoding="UTF-8"?>

<movies>

  <!-- Dark Knight Rises Again. -->
  <movie>
    <movieId>3</movieId>
    <title>Dark Knight Rises Again</title>
    <runtime>165</runtime>
    <releaseDate day="20" month="7" year="2012"/>
    <director>Cristopher Nolan</director>
    <actor>Christian Bale </actor>
    <actor>Morgan Freeman</actor>
    <genre>THR</genre>
    <price>4.99</price>
    <description> Ocho años después de los acontecimientos de The
    Dark Knight, Gotham se encuentra en un estado de paz. En virtud de
    los poderes otorgados por la Ley Dent, el comisario Gordon casi ha
    erradicado la violencia y el crimen organizado. Sin embargo, todavía
    se siente culpable por el encubrimiento de los crímenes de Harvey
    Dent.</description>
  </movie>

  <!-- Con la Muerte en los Talones. -->
  <movie>
    <movieId>4</movieId>
    <title>Con la Muerte en los Talones</title>
    <runtime>136</runtime>
    <releaseDate day="26" month="9" year="1959"/>
    <director>Alfred Hitchcock</director>
    <actor>Cary Grant</actor>
    <actor>Eve Marie Saint</actor>
    <actor>James Manson</actor>
    <genre>THR</genre>
    <genre>COM</genre>
    <price>3.99</price>
    <description> Roger O. Thornhill (Cary Grant) es un ejecutivo
    publicitario de Nueva York al que unos espías confunden con un
    agente del gobierno. Debe escapar, pero lo siguen de cerca. Durante
    la fuga conoce a una atractiva mujer, Eve Kendall (Eva Marie Saint),
    que lo ayuda. Algunas de sus escenas son recordadas por la magnitud
    de las imágenes: la persecución de Thornhill por un campo sembrado
    por una avioneta que quiere matarlo, una pelea en el Monte Rushmore
    y la escena filmada de incógnito en la sede de la ONU en Nueva York.
    </description>
  </movie>
</movies>

```

FORMATO DE UN DOCUMENTO XML

- **Documento XML**
 - Secuencia de caracteres (fichero, secuencia de caracteres que se envía por un socket, etc.) que tiene texto en formato XML
- **Aplicación XML**
 - Conjunto particular de tags que permiten representar determinada información (ej. información sobre películas)
- Un documento XML está **bien formado** ("well-formed") si cumple con el conjunto de reglas siguientes (estas reglas permiten construir parsers eficientes)
 - Se distingue entre mayúsculas y minúsculas
 - Comentarios
 - Empiezan con <!-- y terminan con -->
 - Puede englobar varias líneas
 - Dentro de un comentario no puede aparecer la secuencia --
 - Comentarios tipo <!-- **Comentario** ----> no son válidos

DECLARACIÓN XML

- Declaración XML
 - `<?xml version="1.0" encoding="UTF-8"?>`
 - No es obligatoria pero sí aconsejable
 - Si aparece tiene que ser justo al principio del documento
- version
 - Indica la versión de la especificación XML (no la versión del documento) a la que es conforme el documento
- encoding
 - Indica la codificación de los caracteres del documento
 - Cuando se utiliza XML para intercambio de información entre un cliente y un servidor, lo más normal es usar codificación UTF-8

ELEMENTOS Y ATRIBUTOS

- Todo documento debe tener un elemento raíz (**movies** en el ejemplo)
- Entre el tag de inicio (**<tag>**) de un elemento y el de fin (**</tag>**) puede haber otros elementos o texto
 - "tag" y "elemento" son intercambiables
- No se puede mezclar el orden de los tags anidados
 - El primero elemento que se abre debe ser el último que se cierra
 - Ej. mal formado
 - `<tag-1>`
 - `<tag-2>`
 - `</tag-1>`
 - `</tag-2>`
- Un elemento puede tener atributos
 - El valor del atributo tiene que ir entrecomillado con comillas dobles (") o simples (')
 - Para un elemento dado, un atributo sólo puede tener un valor
- Elemento vacío
 - No tiene elementos anidados ni texto
 - Puede tener atributos
 - Ej.
 - `<tag-1 attr-1="val1" attr-2="val2"></tag-1>`
 - Otra notación más cómoda
 - `<tag-1 attr-1="val1" attr-2="val2"/>`
 - la barra (/) antes del cierre quiere decir que se abre y se cierra a la vez

ELEMENTOS VS ATRIBUTOS

- Existen múltiples alternativas para representar la misma información

```
<movie title="Dark Knight Rises Again" movieId="3" runtime="165"
      releaseDay="20" releaseMonth="7" releaseYear="2012"
      price="4.99">

  <director name="Cristopher Nolan"/>
  <actor name="Christian Bale"/>
  <actor name="Morgan Freeman"/>
  <genre name="THR"/>
  <description>Ocho años después de los acontecimientos de The
  Dark Knight, Gotham se encuentra en un estado de paz. En virtud de
  los poderes otorgados por la Ley Dent, el comisario Gordon casi ha
  erradicado la violencia y el crimen organizado. Sin embargo, todavía
  se siente culpable por el encubrimiento de los crímenes de Harvey
  Dent</description>

</movie>
```

- el title es un atributo del tag movie, en vez de haber un tag title

- Se puede seguir la siguiente convención
 - Usar elementos para datos multivaluados

<code><movie></code>	<code><movie></code>
<code> <actor>C. Bale</actor></code>	<code> <actor name="C. Bale"/></code>
<code> <actor>M. Freeman</actor></code>	<code> <actor name="M. Freeman"/></code>
<code></movie></code>	<code></movie></code>

- Usar contenido de elementos para datos de gran cantidad de texto
 - Usar atributos o contenido de elementos en cualquier otro caso
- Se ha elegido la primera alternativa

REFERENCIAS A ENTIDADES PREDEFINIDAS

- “Referencias a entidades” → permiten incluir ciertos caracteres en el valor de un atributo o en el texto de un elemento

Referencia	Carácter
<code>&lt;</code>	<code><</code>
<code>&amp;</code>	<code>&</code>
<code>&gt;</code>	<code>></code>
<code>&quot;</code>	<code>"</code>
<code>&apos;</code>	<code>'</code>

- Ej.
 - `<tag char=""">Si A < B, entonces ... </tag>`

ESPACIO DE NOMBRES

- Los documentos XML pueden hacer uso de espacios de nombres
- Concepto similar a un paquete Java
- Permiten evitar conflictos de nombre cuando en un documento XML se usan elementos y atributos de distintas aplicaciones XML
- Cada espacio de nombres está asociado a una URI, que debe ser única
 - URI → **identificador de un recurso (típicamente es una URL)**
 - Aconsejable usar URLs (forma fácil de elegir nombres únicos)
 - No tienen por qué tener una existencia real (no suelen tenerla)
- Ej. supuesto
 - Existe una aplicación XML que modela información de críticas de películas
 - Utiliza un espacio de nombres **`http://revies.example.com`**
 - Dispone de varios elementos para modelar la información de la crítica incluyendo los elementos **`title`** y **`description`** que se refieren al título y resumen de la crítica
 - Nuestro servicio decide reutilizar estos elementos para incluir una crítica destacada junto con la información de la película
 - Si ya existe el código que sepa generar/parsear los elementos de ese espacio de nombres, puede reusarse
 - El sitio WEB especializado en cine decide que sus elementos pertenezcan al espacio de nombres **`http://ws.udc.es/movies/xml`** (buena práctica)

```
<?xml version="1.0" encoding="UTF-8"?>

<movies xmlns="http://ws.udc.es/movies/xml"
        xmlns:review="http://reviews.example.com">

    <!-- Dark Knight Rises Again -->

    <movie>
        <movieId>3</movieId>
        <title> Dark Knight Rises Again </title>
        <runtime>103</runtime>
        ...
        <description> Ocho años después ... </description>
        <review:title> Cualquiera puede ser un héroe </review:title>
        <review:description> Es difícil hacer una crítica de una
        película tan compleja ... </review:description>
        <review:rating> 4 </review:rating>
        ...
    </movie>

    ...
</movies>
```

- `<movies xmlns="http://ws.udc.es/movies/xml" xmlns:review="http://reviews.example.com">`
 - `xmlns="http://ws.udc.es/movies/xml"`
 - Espacio de nombres por defecto
 - Todos los elementos contenidos dentro del elemento **movies** que no empiezen por **xxx:**, y el propio elemento **movies**, pertenecen al espacio de nombres **http://ws.udc.es/movies/xml**
 - `xmlns:review="http://reviews.example.com"`
 - Todos los elementos y atributos contenidos dentro del elemento **movies** que empiezan por **review:** pertenecen al espacio de nombres **http://reviews.example.com**
 - **review** actúa sólo como un alias de la URI del espacio de nombres, es decir, el identificador del espacio de nombres no es **review**, sino la URI
 - Conceptualmente se están importando 2 “esquemas”, uno para modelar información sobre películas y otro para modelar información sobre sus críticas

VALIDACIÓN DE DOCUMENTOS XML

- Varios tipos de mecanismos (esquemas) para expresar los elementos y atributos válidos de una aplicación XML y sus restricciones
 - Ej. qué campos y de qué tipo debe contener la información de una película
- Existen 2 estándares de W3C para especificar esquemas XML
 - DTD (Document Type Definition)
 - Sencillo pero menos potente
 - Esquema XML (XML Schema)
 - Complejo
 - Semánticamente muy rico
 - Un esquema XML es a su vez un documento XML
- Documento XML **válido** → documento XML que cumple las restricciones de un esquema (DTD, esquema XML, etc.)
- Aplicación que parsee un documento XML
 - Siempre comprueba que esté **bien formado**
 - **Puede** decidir comprobar si es válido (validar tiene ventajas e inconvenientes)
- En caso de validarlo
 - El documento puede indicar (normalmente a través de URLs) cómo obtener los esquemas que utiliza
 - La aplicación también puede tener copias locales de los esquemas

MOVIES.XSD

- Construcción de un esquema XML (**Movies.xsd**) para el ejemplo de las películas y estudio de los fundamentos básicos de los esquemas XML
- Se asume que
 - Se desea que los elementos pertenezcan al espacio de nombres **http://ws.udc.es/movies/xml**
 - Es buena práctica que cada esquema defina su propio espacio de nombres
- **.xsd** es la extensión que típicamente se utiliza para los esquemas XML (XSD = XML Schema Definition)
- Tutorial de Esquemas XML → <http://www.xfront.com/xml-schema.html>

```
<?xml version="1.0" encoding="UTF-8"?>

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://ws.udc.es/movies/xml"
  xmlns="http://ws.udc.es/movies/xml"
  elementFormDefault="qualified">
```

Para que un documento xml funcione tiene que estar claras cada etiqueta que pertenece al espacio de nombres, en este caso de Movies

```
<!-- "ReleaseDate" type. -->
<xsd:complexType name="ReleaseDate">
  <xsd:attribute name="day" type="xsd:short"/>
  <xsd:attribute name="month" type="xsd:short"/>
  <xsd:attribute name="year" type="xsd:short"/>
</xsd:complexType>
```

```
<!-- "Genre" type.
Possible values of genre:
* COM: Comedy
* DRA: Drama
* HOR: Horror
* ROM: Romance
* SFI: Science fiction
* THR: Thriller
-->
<xsd:simpleType name="Genre">
  <xsd:restriction base="xsd:string">
    <xsd:enumeration value="COM"/>
    <xsd:enumeration value="DRA"/>
    <xsd:enumeration value="HOR"/>
    <xsd:enumeration value="ROM"/>
    <xsd:enumeration value="SFI"/>
    <xsd:enumeration value="THR"/>
  </xsd:restriction>
</xsd:simpleType>
```

```
<!-- "Movie" type. -->
<xsd:complexType name="Movie">
  <xsd:sequence>
    <xsd:element name="movieId" type="xsd:long"/>
    <xsd:element name="title" type="xsd:string"/>
    <xsd:element name="runtime" type="xsd:short"/>
    <xsd:element name="releaseDate" type="ReleaseDate"/>
    <xsd:element name="director" type="xsd:string"
      minOccurs="1" maxOccurs="unbounded"/>
    <xsd:element name="actor" type="xsd:string"
      minOccurs="1" maxOccurs="unbounded"/>
    <xsd:element name="genre" type="Genre"
      minOccurs="1" maxOccurs="unbounded"/>
    <xsd:element name="price" type="xsd:float"/>
    <xsd:element name="description" type="xsd:string"/>
  </xsd:sequence>
</xsd:complexType>
```

```
<!-- "Movies" type. -->
<xsd:complexType name="Movies">
  <xsd:sequence>
    <xsd:element name="movie" type="Movie"
      minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>

<!-- "movies" element (root element). -->
<xsd:element name="movies" type="Movies"/>

</xsd:schema>
```

ESQUEMAS XML

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://ws.udc.es/movies/xml"
  xmlns="http://ws.udc.es/movies/xml"
  elementFormDefault="qualified">
```

- Los elementos y tipos (**schema**, **simpleType**, **short**, **string**, etc.) que se utilizan para definir un esquema XML están definidos en el esquema asociado al espacio de nombres **http://www.w3.org/2001/XMLSchema**
- **targetNamespace** → especifica la URI del espacio de nombres de los elementos definidos en el esquema
- **xmlns="http://ws.udc.es/movies/xml"**
 - Especifica el espacio de nombres por defecto para los elementos y tipos que no se prefijen (típicamente el espacio de nombres especificado en **targetNamespace**)
- **elementFormDefault="qualified"**
 - En los documentos XML que utilicen este esquema, todos los elementos deberán escribirse cualificados (bien porque se ha utilizado el espacio de nombres por defecto o porque se les ha prefijado explícitamente)
 - Es buena práctica especificar siempre **qualified** (por defecto, el valor de **elementFormDefault** es **unqualified**)
- Tipo simple (**simpleType**)
 - Tipos (de elementos o atributos) que tienen sólo valores y no otros atributos o elementos
 - Existen varios tipos predefinidos
 - **string**, **int**, **long**, **short**, **float**, **double**, **boolean**, **byte**, **dateTime**, etc.
- Restricciones de tipos simples
 - Permiten definir un tipo simple a partir de otro restringiendo los valores de este último
 - El tipo **Genre** define un tipo **string** enumerado → sus valores posibles están restringidos a uno de los definidos en la enumeración
 - Otras posibilidades de restricción
 - Definir un tipo derivado de un **string** que restringe la longitud máxima de sus posibles valores
 - Definir un tipo derivado de un **string** que restringe sus valores mediante una expresión regular
- Tipo complejo (**complexType**)
 - Tipos (de elementos) que tienen atributos y/o elementos
 - El tipo **ReleaseDate** define un tipo de elemento que sólo contiene atributos **day**, **month** y **year**, siendo los tres de tipo **short** (simple predefinido)
 - El tipo **Movie** contiene sólo elementos
 - Usa **sequence** para declarar sus elementos
 - **sequence** es un "compositor" que define una secuencia ordenada de elementos
 - Existen otros compositores
 - **all** → los elementos pueden aparecer en cualquier orden → tienen restricciones específicas (ej. los elementos deben tener **maxOccurs="1"**)
 - **choice** → sólo puede aparecer uno de los elementos
 - Cada elemento se define dando
 - Nombre
 - Tipo
 - Opcionalmente, el número mínimo y máximo de ocurrencias posibles → por defecto → **minOccurs="1"** y **maxOccurs="1"**
 - Es posible definir tipos complejos de atributos y elementos
 - La lista de atributos se especifica al final, después del compositor usado para definir los elementos
 - Es posible definir tipos complejos por derivación (se usan restricciones)
- El ejemplo termina definiendo el elemento raíz (**movies**)
- Herramienta online que permite verificar si un documento XML es válido conforme un esquema → <http://www.utilities-online.info/xsdvalidation>

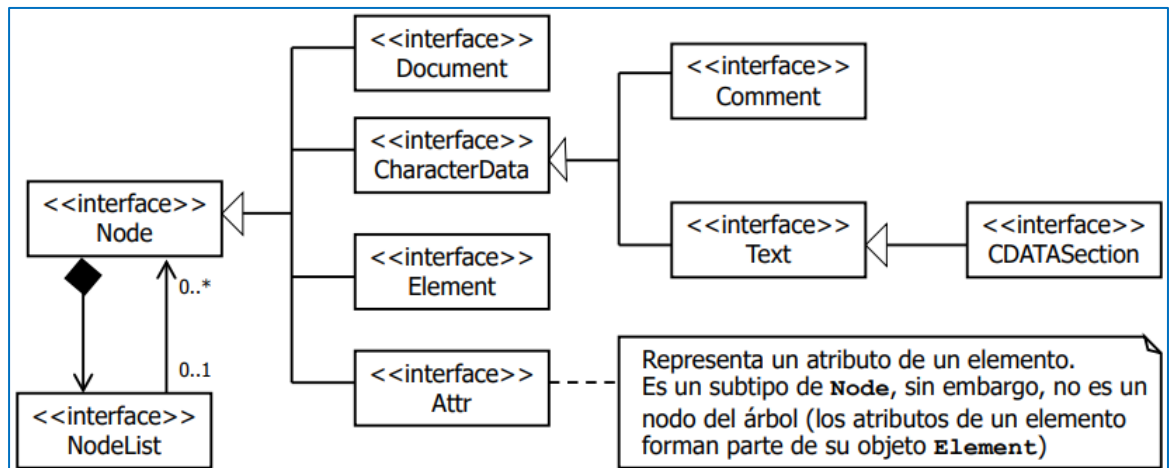
INTRODUCCIÓN

- Un documento XML se apoya en 2 ideas
 - **[Obligatoria]** Tiene que estar bien formado, y en consecuencia, estar construido en base a las normas de XML (los tags se abren y cierran, los tags se anidan de manera jerárquica, los atributos tienen que ir entrecomillados, etc.)
 - **[Opcional]** El conjunto de elementos y atributos, y sus restricciones, pueden estar descritas formalmente en algún tipo de esquema (esquema XML, DTD, etc.) de manera que se pueda comprobar que el documento es **válido**
- Consecuencias
 - Es posible construir parsers genéricos que comprueban que el documento (1) está bien formado, y si se desea, (2) es válido
 - Existen parsers para los lenguajes más usuales
 - En un lenguaje orientado a objetos, un parser XML es una librería de clases
- Tipos de parsers
 - Parsers tipo DOM
 - Parsers tipo “streaming”
- Parsers tipo DOM (Document Object Model)
 - Construyen un árbol en memoria equivalente al documento XML (se puede hacer porque la sintaxis XML sigue un modelo jerárquico)
 - Ventajas
 - Sencillos de utilizar → para acceder a la información del documento sólo hay que recorrer el árbol
 - Suelen permitir modificar/crear un árbol y generar XML a partir de él
 - Desventajas
 - Consumo de memoria alto en aplicaciones servidoras que reciben muchas peticiones que involucran parsear/generar documentos XML grandes (normalmente las aplicaciones servidoras sirven cada petición en un thread)
- Parsers tipo “streaming”
 - Procesan secuencialmente el documento en bloques (no crean un árbol en memoria)
 - Ventajas
 - Mínimo consumo de memoria
 - Especialmente útiles en las situaciones en las que los parsers de tipo DOM son prohibitivos
 - Desventajas
 - No todos tienen soporte para generar XML
 - Más difíciles de usar que los parsers de tipo DOM

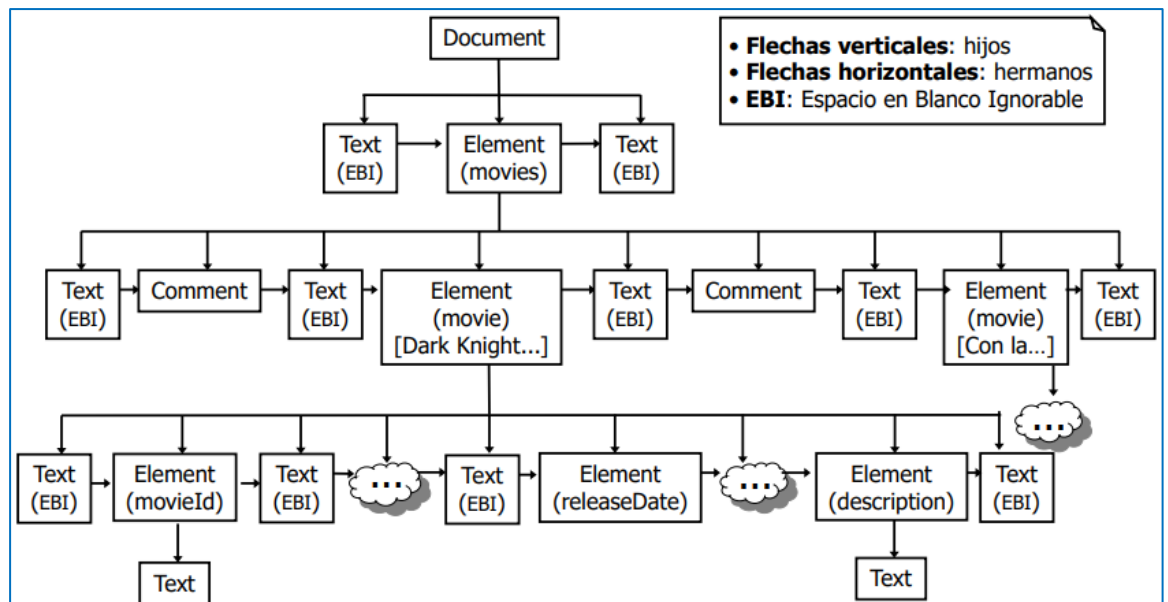
PARSERS JAVA

- SAX (Simple API for XML)
 - Parser tipo streaming
 - Forma parte de Java SE (Standard Edition)
 - API → familia de paquetes **org.xml.sax**
 - Pequeño framework basado en eventos
 - El programador proporciona uno o varios objetos callback a los que el parser llamará cada vez que ocurra un evento de interés (apertura/cierre de un tag, error, etc.)
 - No tiene soporte para generación de XML
 - Disponible para otros lenguajes
- DOM (Document Object Model)
 - Parser tipo DOM
 - La API está estandarizada por el W3C (<http://www.w3.org/>) → disponible para varios lenguajes
 - Forma parte de Java SE
 - API → familia de paquetes **org.w3c.dom**
 - Tiene soporte para generación de XML

- DOM → tipos principales de nodos
 - Paquete **org.w3c.dom**



- DOM → ejemplo
 - Representación DOM de **Movies.xml**



- JDOM (<http://www.jdom.org/>)
 - Alternativa famosa a DOM
 - Open Source
 - Utiliza un modelo similar a DOM
 - Tiene soporte para generación de XML
 - Pensado para el lenguaje Java y más fácil de usar que la API de DOM
 - API de DOM
 - Tediosa de usar → muy bajo nivel (no automatiza aspectos comunes)
 - No está acoplada con el lenguaje Java (ej. no utiliza **java.util.List**, dispone de sus propias listas)

EL LENGUAJE JSON

DEFINICIÓN

- **JavaScript Object Notation (JSON)**
- Basado en un subconjunto de lenguaje de programación JavaScript (estándar ECMA-262)
- Estandarizado en 2013 en el RFC 7158 y ECMA-404
 - Estándar ECMA → describe solamente la sintaxis
 - Estándar RFC → describe sintaxis y algunos aspectos de seguridad e interoperabilidad
 - El último RFC (8259) fue publicado en 2017 y sigue siendo consistente con ECMA-404
- Permite expresar información estructurada y fácilmente parseable
 - Basado en 2 estructuras soportadas por cualquier lenguaje de programación → objetos (registros, structs) y arrays

INFORMACIÓN EN JSON

- Ilustraciones del uso de JSON para expresar información de películas
 - La URL de nuestro servicio devolvería este texto JSON
 - El portal del ejemplo de la introducción de este tema parsearía este texto JSON

```
[
  {
    "movieId": 3,
    "title": "Dark Knight Rises Again",
    "runtime": 165,
    "releaseDate": { "day": 20, "month": 7, "year": 2012 },
    "directors": [ "Cristopher Nolan" ],
    "actors": [ "Christian Bale", "Morgan Freeman" ],
    "genres": [ "THR" ],
    "price": 4.99,
    "description": "Ocho años después de los acontecimientos de The Dark Knight, Gotham se encuentra en un estado de paz. En virtud de los poderes otorgados por la Ley Dent, el comisario Gordon casi ha erradicado la violencia y el crimen organizado. Sin embargo, todavía se siente culpable por el encubrimiento de los crímenes de Harvey Dent."
  },
  {
    "movieId": 4,
    "title": "Con la Muerte en los Talones",
    "runtime": 136,
    "releaseDate": { "day": 26, "month": 9, "year": 1959 },
    "directors": [ "Alfred Hitchcock" ],
    "actors": [ "Cary Grant", "Eve Marie Saint", "James Manson" ],
    "genres": [ "THR", "COM" ],
    "price": 3.99,
    "description": "Roger O. Thornhill (Cary Grant) es un ejecutivo publicitario de Nueva York al que unos espías confunden con un agente del gobierno. Debe escapar, pero lo siguen de cerca. Durante la fuga conoce a una atractiva mujer, Eve Kendall (Eva Marie Saint), que lo ayuda."
  }
]
```

FORMATO DE UN DOCUMENTO JSON

- **Documento JSON**
 - Secuencia de caracteres (archivo, secuencia de caracteres que se envía por un socket, etc.) que tiene texto en formato JSON
 - Codificación de caracteres UTF-8

- Documento JSON **bien formado** → documento JSON que cumple con las reglas siguientes
 - Estas reglas permiten construir parsers eficientes
- Un documento de JSON contiene un valor que puede ser de cualquiera de los siguientes 6 tipos
 - Objeto
 - Array
 - Cadena (string)
 - Número
 - Booleano → *true* o *false*
 - *null*
- Se distingue entre mayúsculas y minúsculas
- Objeto
 - Se especifica entre llaves → {}
 - Conjunto sin orden de pares nombres de campo – valor
 - Los nombres de campos son cadenas (strings)
 - Los valores pueden ser de cualquier tipo JSON válido
 - Los nombres de campos se separan de los valores por 2 puntos
 - Los pares nombre de campo – valor se separan por comas
- Array
 - Se especifican entre corchetes → []
 - Colección ordenada de valores
 - Pueden ser de cualquier tipo válido
 - Se separan por comas
- Cadena (string)
 - Se especifican entre comillas dobles
 - Cualquier carácter Unicode excepto “ o \ o caracteres de control
 - No pueden contener caracteres de retorno de carro, nueva línea, tabulador, etc.
 - El carácter \ se utiliza para
 - Escapar “ o \
 - Especificar retornos de carro (\r), nueva línea (\n), tabulador (\t), etc.
 - Especificar un carácter a partir de su código Unicode → \u + 4 dígitos (hexadecimal)
- Consideraciones respecto a XML
 - No hay comentarios
 - No existe el concepto de atributo
 - No existe el concepto de espacio de nombres
 - Los objetos y arrays “no tienen nombre” si no son el valor de un campo de un objeto
 - En XML cuando se tiene un elemento con un único subelemento no se puede distinguir si se trata de un array o no (si no se tiene un esquema contra el que validarlo)

VALIDACIÓN DE DOCUMENTOS JSON

- Los esquemas JSON (JSON Schema) permiten expresar restricciones sobre el contenido de un documento JSON
 - <https://json-schema.org>
 - Está en procesos de estandarización
 - Ej. especificar con qué tipo JSON se va a representar la información de una película o de una lista de películas
- Documento JSON **válido** → cumple las restricciones de un esquema JSON
- Aplicación que parsea un documento JSON
 - Siempre comprueba que esté **bien formado**
 - Puede decidir comprobar si es **válido** (validar tiene ventajas e inconvenientes)
- Los esquemas JSON son a su vez documentos JSON
- No existe una forma estandarizada para que un documento indique cómo obtener el esquema que utiliza (en XML sí)

MOVIESSCHEMA.JSON

- Construcción de un esquema JSON (**MoviesSchema.json**) para el ejemplo de las películas y estudio de los fundamentos básicos de los esquemas JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "http://ws.udc.es/movies/json",
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "movieId": {"type": "integer"},
      "title": {"type": "string"},
      "runtime": {"type": "integer"},
      "releaseDate": {
        "type": "object",
        "properties": {
          "day": {"type": "integer"},
          "month": {"type": "integer"},
          "year": {"type": "integer"}
        },
        "additionalProperties": false,
        "required": ["day", "month", "year"]
      },
      "directors": {
        "type": "array",
        "items": {"type": "string"},
        "minItems": 1
      },
      "actors": {
        "type": "array",
        "items": {"type": "string"},
        "minItems": 1
      },
      "genres": {
        "type": "array",
        "items": { "enum": ["COM", "DRA", "HOR", "ROM", "SFI", "THR"] },
        "minItems": 1
      },
      "price": {"type": "number"},
      "description": {"type": "string"}
    },
    "additionalProperties": false,
    "required": ["movieId", "title", "runtime", "releaseDate", "directors", "actors", "genres", "price", "description"]
  }
}
```

ESQUEMAS JSON

- Contienen un objeto con campos que imponen un conjunto de restricciones
 - Esquema más simple → ({ }) → no impone ninguna restricción (cualquier documento JSON es válido según este esquema)
- Campos solamente descriptivos y no expresan restricciones
 - **\$schema**
 - Indica que el documento JSON es un esquema, y además puede indicar conforme a qué versión
 - Buena práctica aunque no obligatorio

- **\$id**
 - Para asignar un identificador único al esquema
 - Buena práctica aunque no obligatorio
- **\$comment**
 - Para añadir comentarios
- Campo **type**
 - Restringe los documentos válidos de uno o varios de los tipos existentes en JSON (para numéricos hay 2 tipos diferentes)
 - Valores
 - Cadena especificando el nombre de un tipo
 - Numérico (**integer** o **number**), **string**, **object**, **array**, **boolean**, **null**
 - Ej. → `{"type": "string"}`
 - Array de cadenas especificando varios nombres de tipos → pasaría la validación si fuese de cualquier de los tipos
 - Ej. → `{"type": ["string", "null"]}`
 - El uso del tipo **null** en combinación con cualquier otro tipo es útil para valores que pueden ser nulos
- Tipos “básicos”
 - String (`{"type": "string"}`)
 - Restricciones
 - Longitud máxima (**maxLength**) y mínima (**minLength**)
 - Patrón (**pattern**)
 - Formatos o patrones predefinidos (**format**): **date**, **time**, **date-time**, **email**
 - Tipos numéricos (`{"type": "integer"}`) o (`{"type": "number"}`)
 - Dos tipos → **integer** (enteros) y **number** (enteros o decimales)
 - Restricciones
 - Múltiplos (**multipleOf**)
 - Rango (**maximum**, **minimum**, **exclusiveMaximum**, **exclusiveMinimum**)
 - Boolean (`{"type": "boolean"}`)
- Tipo array (`{"type": "array"}`)
 - Por defecto los elementos pueden ser de cualquier tipo
 - Se usa el campo **items** para restringir el tipo de los elementos
 - Se especifica como un objeto que a su vez es un JSON schema
 - Se puede usar **minItems** y **maxItems**
- Tipo object (`{"type": "object"}`)
 - Por defecto, cualquier objeto es válido
 - Se usa el campo **properties** para restringir las propiedades que puede tener el objeto
 - El objeto se compone de campos donde los nombres de estos especifican las propiedades y los valores asociados a cada campo representan las restricciones de dichas propiedades, siendo a su vez objetos que siguen un esquema JSON
 - Por defecto, si el objeto tiene esas propiedades tiene el tipo adecuado
 - **additionalProperties** → se utiliza para especificar si el objeto puede tener o no propiedades que no estén en el esquema
 - **required** → se utiliza para especificar qué propiedades son obligatorias
- Enumerados
 - Se especifican con el campo **enum**
 - El valor es un array con al menos un elemento y sus valores deben ser únicos → pueden ser de diferentes tipos
- Herramienta online que permite verificar si un documento JSON es válido conforme un esquema → <https://jsonschema.dev/>

PARSIN DE DOCUMENTOS JSON

INTRODUCCIÓN

- Documento JSON apoyado sobre 2 ideas
 - **[Obligatoria]** Tiene que estar bien formado, y en consecuencia, estar construido en base a las normas de JSON
 - **[Opcional]** El contenido del documento puede estar descrito formalmente en un esquema JSON de manera que se pueda comprobar que el documento es **válido**
- Consecuencias
 - Es posible construir **parsers** genéricos que comprueban que el documento está bien formado, y **validadores** genéricos para comprobar, si se desea, que es válido
 - Existen parsers y validadores para los lenguajes más usuales
- Tipos de parsers
 - Parsers tipo “Modelo de árbol” o “Modelo de objetos” (equivalentes a los parsers XML tipo DOM)
 - Parsers tipo “streaming” (equivalentes a los parsers XML tipo straming)
- Parsers tipo modelo de árbol o modelo de objetos
 - Construyen un árbol en memoria equivalente al documento JSON (pueden hacerlo, dado que JSON se basa en arrays y objetos/registros)
 - Mismas ventajas y desventajas que los parsers XML de tipo DOM
- Parsers tipo “streaming”
 - Procesan secuencialmente el documento en bloques (no construyen un árbol en memoria)
 - Mismas ventajas y desventajas que los parsers XML de tipo streaming

PARSERS JAVA

- JSON-P (JSON Processing)
 - API estándar para parsear, generar, transformar y consultar documentos JSON
 - Incluye tanto un parser tipo streaming como tipo modelo de árbol
 - Forma parte de Java EE / Jakarta EE
 - Existen pocas implementaciones
- Existen multitud de librerías alternativas al estándar que implementan parsers JSON de ambos tipos
 - Jackson, GSON, etc.
 - Mucho más utilizas
 - Ej. en la asigntura se utiliza Jackson y el parser tipo modelo de árbol

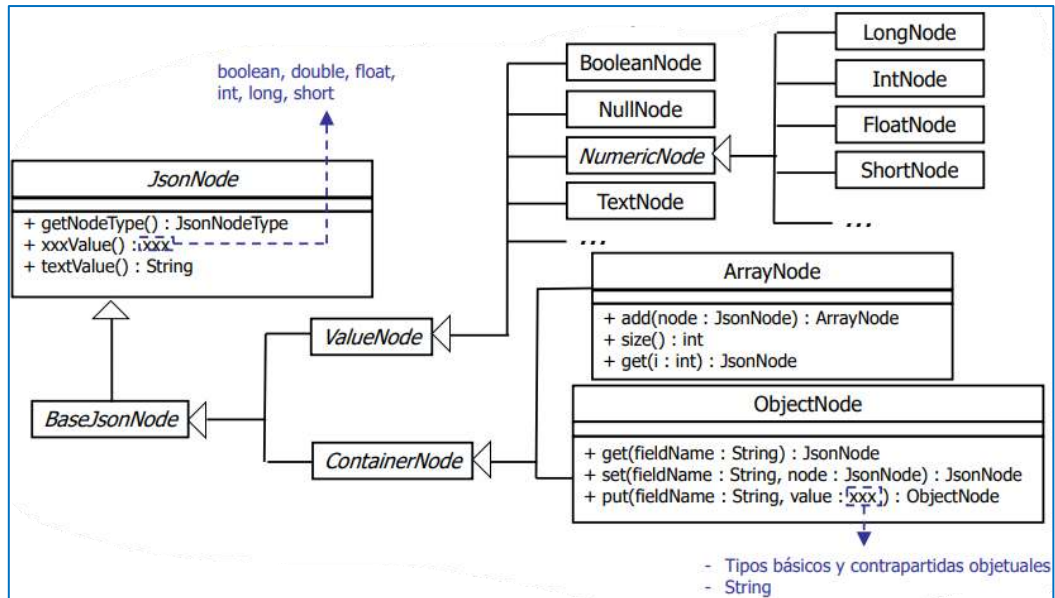
VALIDADORES JAVA

- No existen APIs estándar para validar que un documento JSON es conforme a un esquema JSON
- Librerías que implementan validadores
 - JSON Schema Validator → <https://github.com/java-json-tools/json-schema-validator>
 - everit.org/json.schema → <https://github.com/everit-org/json-schema>
 - Justify (usa JSON-P) → <https://github.com/leadpony/justify>
 - Librerías para diferentes lenguajes → <https://json-schema.org/implementations.html>

JACKSON

- Cosas que contiene el módulo jackson-databind
 - Clases que modelan los distintos tipos de nodos de un árbol Jackson
 - Clases que permiten construir un árbol Jackson a partir de un documento JSON en formato texto
 - Clases que permiten construir un documento JSON en formato texto a partir de un árbol Jackson
 - Paquete **com.fasterxml.jackson.databind**
 - Utiliza internamente el módulo jackson-core, que define
 - La API de streaming
 - Abstracciones básicas compartidas por diferentes módulos

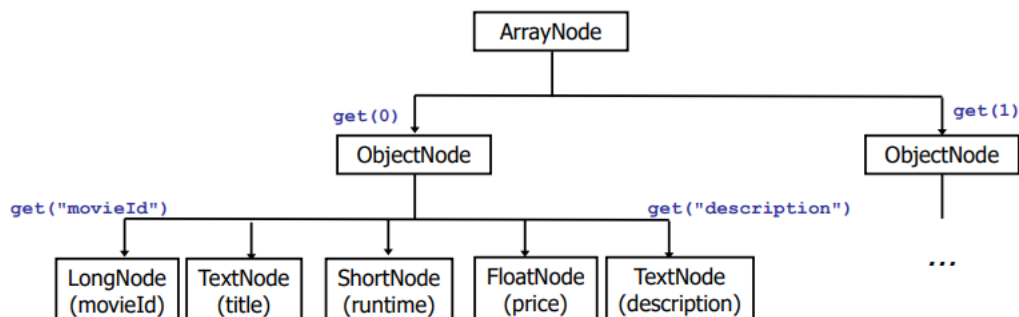
- Visión global de **com.fasterxml.jackson.databind** y subpaquetes → principales tipos de nodos y métodos



- Ej. documento JSON del ejemplo ws-movies
 - Para cada película
 - No se expone fecha de creación (el diseñador ha decidido que no es relevante para aplicaciones externas)
 - Elemento **movieId** es opcional

```
[
  {
    "movieId": 3,
    "title": "Dark Knight Rises Again",
    "runtime": 165,
    "price": 4.99,
    "description": "Ocho años después de ... "
  },
  {
    "movieId": 4,
    "title": "Con la Muerte en los Talones",
    "runtime": 136,
    "price": 3.99,
    "description": " Roger O. Thornhill (Cary Grant) es un ... "
  }
]
```

- Representación del documento JSON

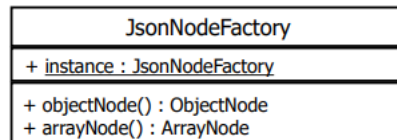


- Comentarios
 - Los métodos **xxxValue** de **JsonNode** no lanzan una excepción si el valor del nodo no se puede convertir al tipo correspondiente
 - Devuelven un valor por defecto
 - Ej. 0 para los tipos numéricos, **null** para String y byte[] y **false** para boolean
 - Los métodos **put** de **ObjectNode** mostrados en el diagrama
 - Crean el nodo adecuado según el tipo del valor que reciben
 - Devuelven la misma instancia sobre la que se invocan, lo que permite “ecadenar” llamadas
 - **JsonNodeType** es un enumerado que tiene un valor por cada tipo de nodo

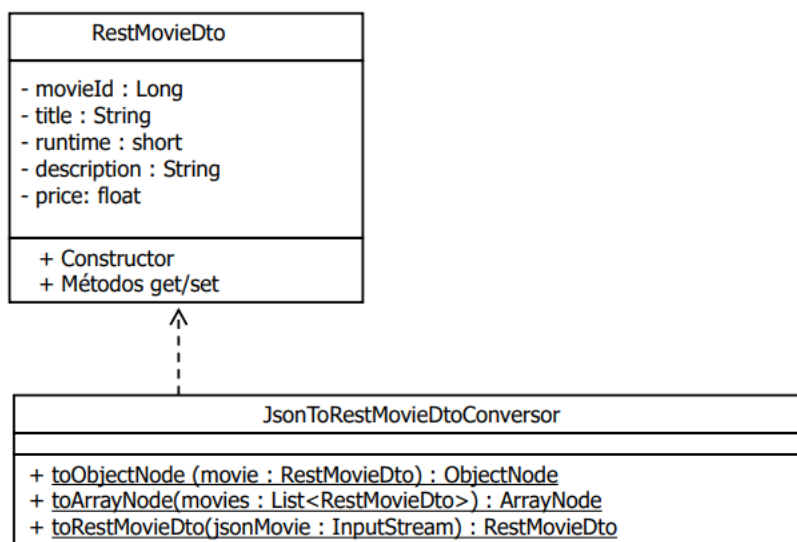
- Visión global de **com.fasterxml.jackson.databind** → Transformaciones texto JSON ↔ árbol Jackson



- El método **readTree** parsea la entrada y devuelve el nodo raíz del árbol
- El método **writer** se invoca pasándole una instancia de **DefaultPrettyPrinter** para que devuelva una instancia de **ObjectWriter** que genere JSON en formato “bonito”
 - El método **writeValue** escribe el JSON correspondiente al objeto que recibe (en este caso **JsonNode**, correspondiente al nodo raíz del árbol)
- Visión global de **com.fasterxml.jackson.databind** → Creación de nodos objeto y array vacíos



- Dado que no tiene estado se puede usar la misma instancia siempre
 - Se obtiene a través del atributo **instance**
- Ej. parte de la clase **JsonToRestMovieDtoConversor** de los ejemplos de la asignatura
 - Dispone de métodos para convertir instancias de **RestMovieDto** a JSON y viceversa
 - **toObjectNode (RestMovieDto) : ObjectNode**
 - Recibe un objeto **RestMovieDto** y genera un árbol Jackson con la misma información
 - **toArrayNode (List<RestMovieDto>) : ArrayNode**
 - Recibe una lista de objetos **RestMovieDto** y genera un árbol Jackson con la misma información (delega en el método anterior)
 - **toRestoMovieDto (InputStream) : RestMovieDto**
 - Recibe el texto de un documento JSON que tiene la información de una película y crea un objeto **RestMovieDto** con la misma información



```

package es.udc.ws.movies.restservice.json;

...
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ArrayNode;
import com.fasterxml.jackson.databind.node.JsonNodeFactory;
import com.fasterxml.jackson.databind.node.JsonNodeType;
import com.fasterxml.jackson.databind.node.ObjectNode;

import es.udc.ws.movies.restservice.dto.RestMovieDto;
import es.udc.ws.util.json.ObjectMapperFactory;
import es.udc.ws.util.json.exceptions.ParsingException;

public class JsonToRestMovieDtoConversor {

    public static ObjectNode toObjectNode(RestMovieDto movie) {

        ObjectNode movieObject = JsonNodeFactory.instance.objectNode();

        movieObject.put("movieId", movie.getMovieId()).
            put("title", movie.getTitle()).
            put("runtime", movie.getRuntime()).
            put("price", movie.getPrice()).
            put("description", movie.getDescription());

        return movieObject;
    }

    public static ArrayNode toArrayNode(List<RestMovieDto> movies) {

        ArrayNode moviesNode = JsonNodeFactory.instance.arrayNode();
        for (int i = 0; i < movies.size(); i++) {
            RestMovieDto movieDto = movies.get(i);
            ObjectNode movieObject = toObjectNode(movieDto);
            moviesNode.add(movieObject);
        }

        return moviesNode;
    }

    public static RestMovieDto toRestMovieDto(InputStream jsonMovie) throws
        ParsingException {
        try {
            ObjectMapper objectMapper = ObjectMapperFactory.instance();
            JsonNode rootNode = objectMapper.readTree(jsonMovie);
            if (rootNode.getNodeType() != JsonNodeType.OBJECT) {
                throw new ParsingException("Unrecognized JSON (object expected)");
            } else {
                ObjectNode movieObject = (ObjectNode) rootNode;
                JsonNode movieIdNode = movieObject.get("movieId");
                Long movieId =
                    (movieIdNode != null) ? movieIdNode.longValue() : null;
                String title = movieObject.get("title").textValue().trim();
                String description =
                    movieObject.get("description").textValue().trim();
                short runtime = movieObject.get("runtime").shortValue();
                float price = movieObject.get("price").floatValue();

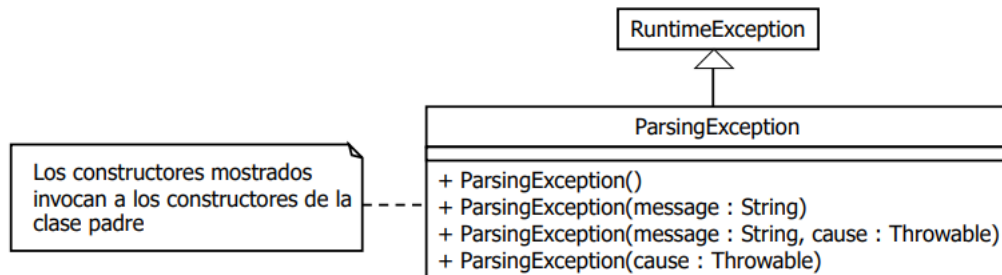
                return
                    new RestMovieDto(movieId, title, runtime, description, price);
            }
        } catch (ParsingException ex) {
            throw ex;
        } catch (Exception e) {
            throw new ParsingException(e);
        }
    }
}

```

COMENTARIOS

- **es.udc.ws.util.json.exceptions.ParsingException**

- Los métodos públicos de la clase **JsonToRestMovieDtoConversor** devuelven una excepción cuando hay algún problema durante el proceso de parsing
- Forma parte del subsistema de utilidades de los ejemplos de la asignatura
- Extiende a **RuntimeException**
 - Representa un error grave
 - Sólo es preciso capturarla en los lugares en los que explícitamente se quiere tratar
 - En el resto de los sitios, la excepción “fluye” hacia arriba
- Como cualquier tipo de excepción (**java.lang.Throwable**), permite especificar un mensaje y/o encapsular una excepción (la que produjo el problema)



- **es.udc.ws.util.json.ObjectMapperFactory**

- Forma parte del subsistema de utilidades de los ejemplos de la asignatura
- Clase utilidad que devuelve siempre la misma instancia de **ObjectMapper**
 - **ObjectMapper** es thread-safe